EnsGuard: federated defense against low-rate flow-table overflow

Status report for the supervisor: architecture, experiments, results, strengths, weaknesses and the next tests. 6 November 2026.

The short version

- **What we built.** EnsGuard lets several SDN controllers share trained detectors instead of raw traffic. Each controller scores every flow-table entry every second, evicts suspicious entries, blocks hosts that keep getting evicted, and falls back to a fair share of the table per port.
- **Clear win (offline, simulator).** EnsGuard is the best detector at recognising attacks a controller never trained on. At a 0.1 % false-alarm rate it catches 98.7 % of attacks seen only by another controller, where the published detectors fall to 79-82 %. It catches 48.5 % of an attack no controller saw, against 34.4 % for our earlier FL-DualGuard and 9.5-24.9 % for FTOP, FTMaster, FloRa and DMS-BTT.
- **Real testbed (preregistered, 200 runs).** EnsGuard beats the classic defenses (EVICT, SIFT, TableGuard) on how much of the table the attacker holds: 14 of 15 paired runs each, Holm-adjusted p = 0.004. It does **not** beat the modern ML defenses (DMS-BTT, FTMaster, FTOP, rDefender, FL-DualGuard) on harm to normal users or on the attacker's share. It blocked an innocent host in 4 of its 20 runs.
- **Why the testbed is harder.** Per-entry logs recorded on the testbed show that the simulator already predicts the problem. On the real trace traffic (bigFlows), the disguised attacks A4 and E9P score lower than the noisiest normal traffic, so *every* detector misses them, ours and the published ones. Poll jitter on the testbed is not the cause.
- **A fix we tried, with rules fixed in advance, failed.** Adding the trace domain as a fourth federated controller (trained on its normal traffic plus one standard attack) did not catch A4. It also cut detection of unseen attacks from 48.5 % to 14.9 %. We stopped as the rules required; this is a useful negative result.
- **Proposed paper.** An honest paper: federated detection that generalises offline, a preregistered testbed comparison with eight other defenses, the measured simulator-to-testbed gap, and the negative result. Draft: docs/paper_optionA.md.

How to read this report

Section 1 explains the attack. Section 2 walks through the architecture step by step. Section 3 describes how we tested. Section 4 gives the results with figures. Section 5 lists where EnsGuard does well and where it needs work. Section 6 proposes the next tests. Section 7 compares our experiments with the related papers. The appendix has the remaining figures and a glossary.

1. The problem: low-rate flow-table overflow (LOFT)

An SDN switch keeps a **flow table**: a list of rules saying where each connection's packets go. Our tables hold 3,500 rules. A rule disappears after 10 s without traffic (the *idle timeout*). When a packet has no rule, the switch asks the controller (a **Packet-In**), and the controller installs a new rule.

A LOFT attacker opens many fake connections and sends one small packet on each just before its 10 s timeout. Each fake rule stays alive at almost no traffic cost. The table slowly fills; when it is full, normal users' new connections are refused or delayed. Because the traffic rate is tiny, rate-based DDoS alarms do not fire.

Attack variants we test

Variant	How it behaves	Closest attack in the literature
A2 (standard)	100 new rules per burst, a burst every 5.5-9 s, one 64-byte packet per rule, from the attacker's own address	The original LOFT attack; the attack groups of the Tang et al. papers and DMS-BTT
A4 (mimicry)	Same speed as A2, but 1-3 packets per refresh sized like normal traffic, from a pool of addresses	rDefender's advanced low-rate attack (trace replay)
E9P (adaptive)	Refreshes every 2-3 s with normal-looking packets; the best attack our search found against six of our seven defenses	ALFO's adaptive attack
A1, A3, A5, A6, A7	Other speeds, spoofed sources, a slow fill (A6, never trained on by anyone) and an on/off pulse	Used in the simulator

Important. A2 is the attack the other papers test. A4 and E9P are harder, disguised versions that only two of the twelve related papers (rDefender and ALFO) test in any form. Our results on A4 and E9P therefore go beyond what the other papers report, and cannot be compared with their published numbers.

2. EnsGuard architecture, step by step

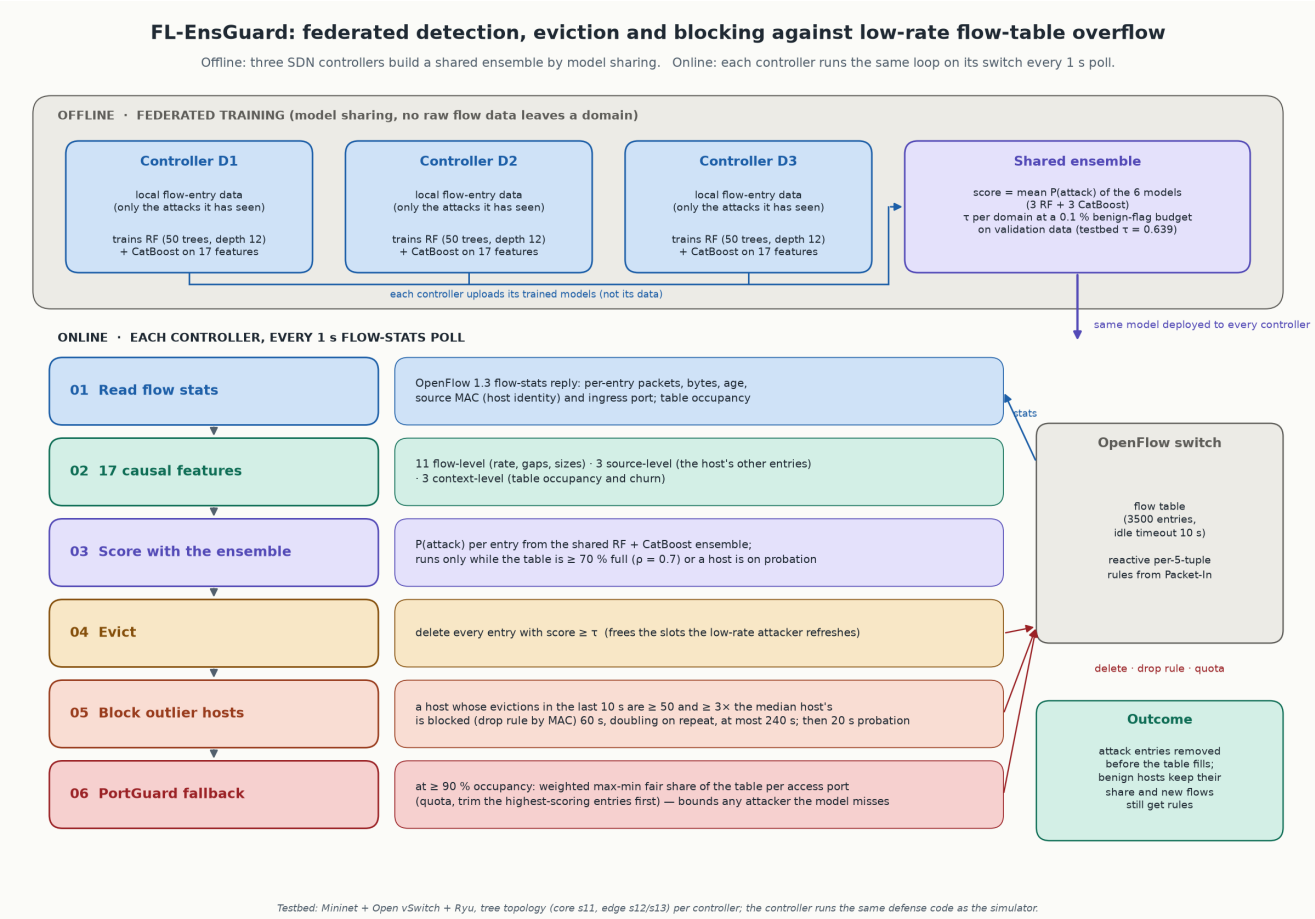


Figure A. EnsGuard: offline federated training (top) and the loop each controller runs every 1 s poll (bottom).

Offline: building the shared detector

- **Step 1. Each controller trains locally.** Controllers D1, D2 and D3 each collect labelled flow-table entries from their own network. Each has seen different attacks: D1 saw A1 and A4, D2 saw A2 and A5, D3 saw A3 and A7. Nobody saw A6 or E9P.
- **Step 2. Two models per controller.** Each trains a Random Forest (50 trees, depth 12) and a CatBoost model on the same 17 features.
- **Step 3. Models are shared, data is not.** Each controller uploads its trained models; the shared detector is the average attack probability of all 6 models. A controller benefits from attacks only the others have seen.
- **Step 4. Each controller sets its own alarm threshold τ.** τ is chosen so that only 0.1 % of that controller's own normal entries would be flagged (on the testbed's traffic, τ = 0.639).

Online: the loop every second on every switch

- **01 Read flow statistics** from the switch: per-entry packets, bytes, age, source MAC and port, and how full the table is.
- **02 Compute 17 features** per entry: 11 about the flow (rate, gaps between packets, sizes), 3 about its source host, 3 about the table (occupancy, churn).
- **03 Score** each entry with the shared ensemble (only while the table is at least 70 % full, or a host is on probation).
- **04 Evict** every entry scoring at least τ.
- **05 Block outlier hosts.** A host whose evictions in the last 10 s are at least 50 and at least 3 times the median host's is blocked for 60 s (doubling on repeat, at most 240 s).

- **06 PortGuard fallback.** If the table is still at least 90 % full, each access port gets a weighted fair share, trimming the highest-scoring entries first. This bounds any attacker the detector misses.

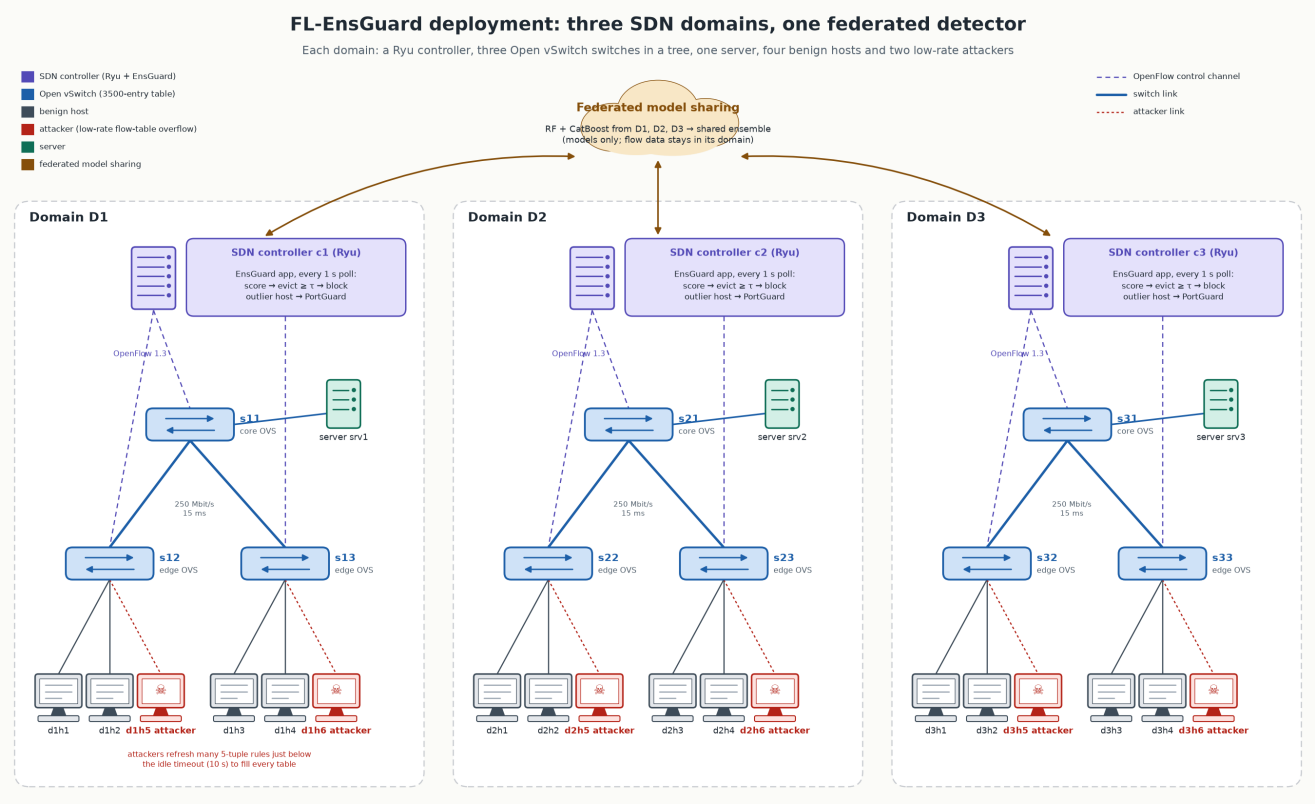


Figure B. Deployment: each domain has a Ryu controller, a core switch and two access switches, four normal hosts and two attackers. On the testbed laptop we run one domain at a time (see Section 3).

Why this design

- **Federation** addresses the main gap in the literature: per-controller detectors are trained and tested on the same attacks and fail on attacks seen elsewhere.
- **Two model families** (RF and CatBoost) were chosen on validation data because their average generalised slightly better than either alone (Section 4.1 discusses the cost).
- **Evict, then block, then fair share** mirrors the published systems (FTMaster, FTODefender and FloRa evict and block) but adds a fallback that does not depend on the detector.

3. How we tested

Three kinds of experiment

Experiment	Where	What it answers
E11: same-data detection benchmark	Simulator data (domains D1-D3), offline	Which detector recognises attacks it did not train on? Every detector is trained and scored on identical rows.
Simulator online test	Our simulator LOFT-Bench, fresh seeds 26000-26004, run once	How well does the full defense protect users when everything is simulated?
Preregistered testbed campaign (v3)	Real Mininet + Open vSwitch + Ryu testbed, seeds 4120-4124	Does EnsGuard beat 8 other defenses on a real OpenFlow controller? 200 runs.
Testbed entry logs	12 extra testbed runs with per-entry logging, seeds 4131-4133	Why do testbed and simulator disagree?
v4 trial	Simulator + the 12 testbed logs	Does adding the trace controller to the federation fix the miss?

Testbed setup

- One 4-core laptop; Mininet, Open vSwitch, Ryu with OpenFlow 1.3; one controller per run.
- Topology: core switch s11 to the server, access switches s12 and s13 with 3 hosts each, one attacker per access switch. All tables 3,500 rules, idle timeout 10 s, all monitored.
- Normal traffic: the real bigFlows trace (as in DMS-BTT), replayed at half speed. The speed was fixed in advance as the fastest at which a no-attack run loses at most 0.25 % of first packets.
- Each run: 300 s, attack starting at 50 s. A probe opens a new TCP connection every 0.5 s to measure what a new user experiences.
- Ten defenses: none, EVICT, SIFT, TableGuard, rDefender, FTMaster, FTOP, DMS-BTT, FL-DualGuard (our earlier system) and EnsGuard. Four traffic types (A2, A4, E9P, normal only) x 5 seeds = 200 runs.

Preregistration (fixed before any defense ran on the test seeds)

Hypothesis	Statement	Test
H1	EnsGuard harms fewer normal flows than each of the 8 comparators	one-sided exact sign test over 15 paired attack runs, ties count against, Holm correction over the 8
H2	EnsGuard keeps a lower average attacker share of the core table than each of the 8	same
H3	With no attack: mean harm at most 0.5 % and lost first packets at most 1 %	descriptive
H4	No normal host is ever blocked in EnsGuard's 20 runs	descriptive

Main measures

- **Harm to users:** normal flows harmed (refused, dropped or first packet lost); lost first packets at the controller; new-connection success and connect time.
- **Table measures used by the other papers:** attacker share of the table (Pa, average and maximum); table usage; seconds the table was full; Packet-In and Table-Full message counts; processing time; CPU and memory.
- **Detection:** accuracy, precision, recall, F1, false-positive rate, PR-AUC.

4. Results so far

4.1 Offline detection (E11): EnsGuard's clearest win

Recall at a 0.1 % false-alarm rate, by whether the scoring controller trained on the attack (results/e11_detect/table_main.csv).

Detector	Attack seen by this controller	Seen only by another controller	Seen by no controller	PR-AUC, seen by none
FTOP	92.0 %	79.4 %	9.5 %	42.2
FTMaster	92.4 %	81.3 %	15.0 %	43.6
FloRa	97.5 %	82.2 %	24.9 %	55.0
DMS-BTT	92.3 %	81.6 %	14.3 %	39.6
FL-DualGuard (ours, earlier)	97.4 %	96.7 %	34.4 %	93.3
EnsGuard	99.2 %	98.7 %	48.5 %	92.7

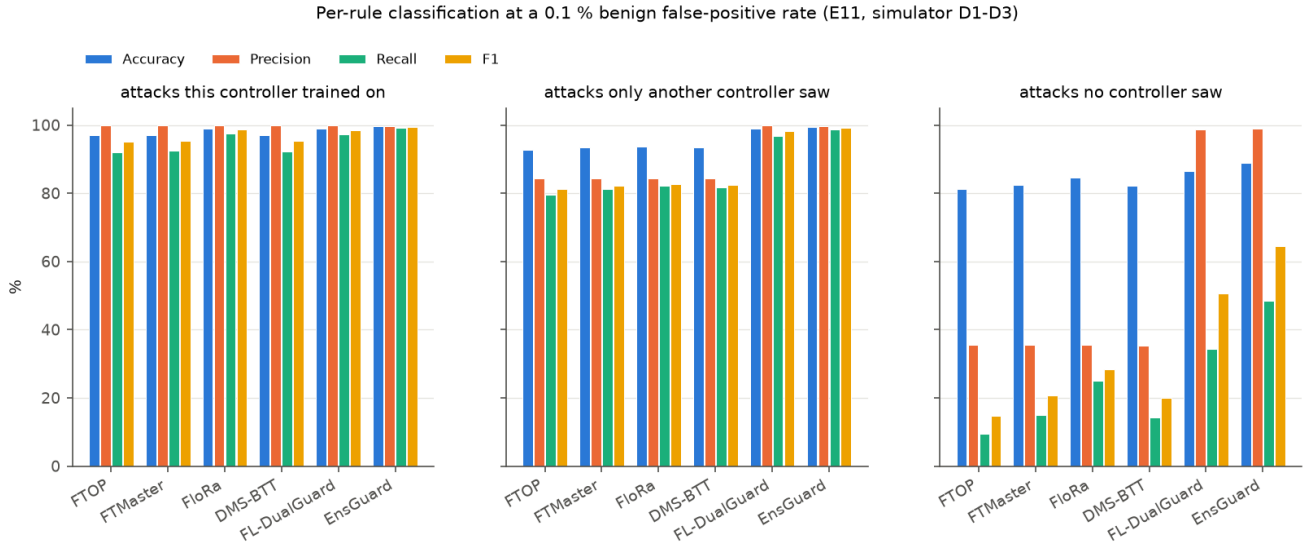


Figure 9. Accuracy, precision, recall and F1 by attack exposure. All detectors are near-perfect on attacks they trained on; only the federated ones keep it on attacks seen elsewhere.

Cost of the two-model ensemble (results/e1_pair/cost.csv): scoring 10,000 entries takes 71.8 ms with the RF and 18.9 ms with CatBoost. The RF is about 80 % of EnsGuard's scoring time, but alone it is the weakest at new attacks (F1 77.3 against 84.0 for CatBoost alone and 84.3 for both). CatBoost alone keeps almost all the benefit at about a fifth of the cost; this is a candidate improvement (Section 6).

4.2 Simulator online test: the best system in simulation

Attack	EnsGuard harm	EnsGuard attacker share	For comparison
A2	0.36 %	2.0 %	
A4	0.73 %	5.0 %	DMS-BTT with blocklist: harm 20.5 %, share 39.3 %
A6 (unseen)	0.37 %	5.3 %	
E9P	2.0 %	26.2 %	FL-DualGuard 1.1 % harm; without PortGuard EnsGuard's harm is 37.7 %
normal only	0.08 %	0	no normal host blocked

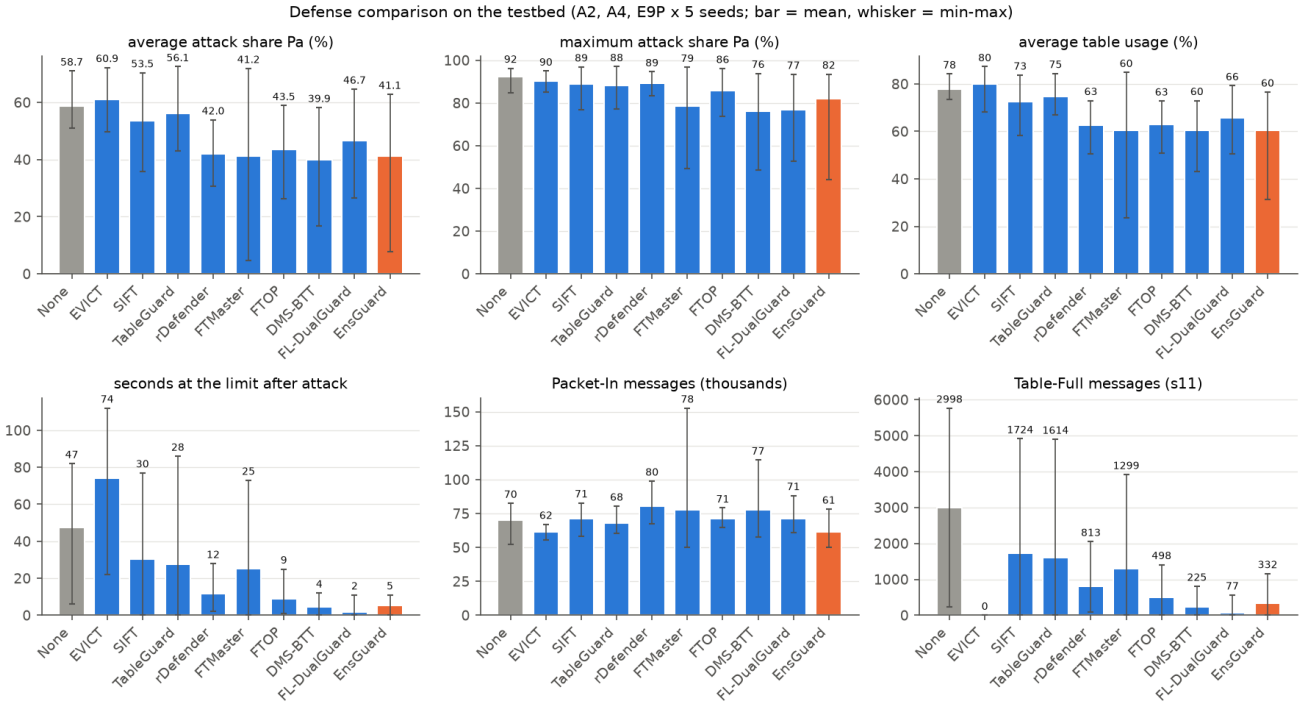
Source: results/evictblock/test_summary.csv (domains D1-D3, fresh seeds 26000-26004, run once).

4.3 Testbed: the preregistered hypotheses

Hypothesis	Result	Detail
H1 lower harm	Not supported against any of the 8	EnsGuard mean harm 3.76 %; DMS-BTT 3.31 %, FL-DualGuard 4.06 %, FTMaster 4.60 %, FTOP 5.14 %, EVICT 5.19 %, rDefender 6.16 %, TableGuard 8.93 %, SIFT 9.49 %
H2 lower attacker share	Supported against EVICT, SIFT, TableGuard only	14 of 15 wins each, Holm p = 0.004. EnsGuard 41.2 % against DMS-BTT 39.9 %, FTMaster 41.2 %, rDefender 42.0 %, FTOP 43.5 %, FL-DualGuard 46.7 %
H3 clean without attack	Passes	harm 0.00 %, lost first packets 0.04 %
H4 no innocent host blocked	Fails	a normal host was blocked in 4 of 20 runs

Source: results/testbed_prereg_v3/report.md.

4.4 Testbed comparison in the other papers' format



EVICT replaces entries when the table is full, so the switch sends no Table-Full message (the papers report NA).

Figure 3. The comparison table of FTMaster, FTODefender, FTOP and DMS-BTT, measured on our testbed (A2, A4, E9P x 5 seeds; bar = mean, whisker = min-max; EnsGuard in orange). EnsGuard is in the leading group on average attacker share and table usage, and has the fewest Packet-Ins, but no clear overall win.

4.5 How the table fills over time

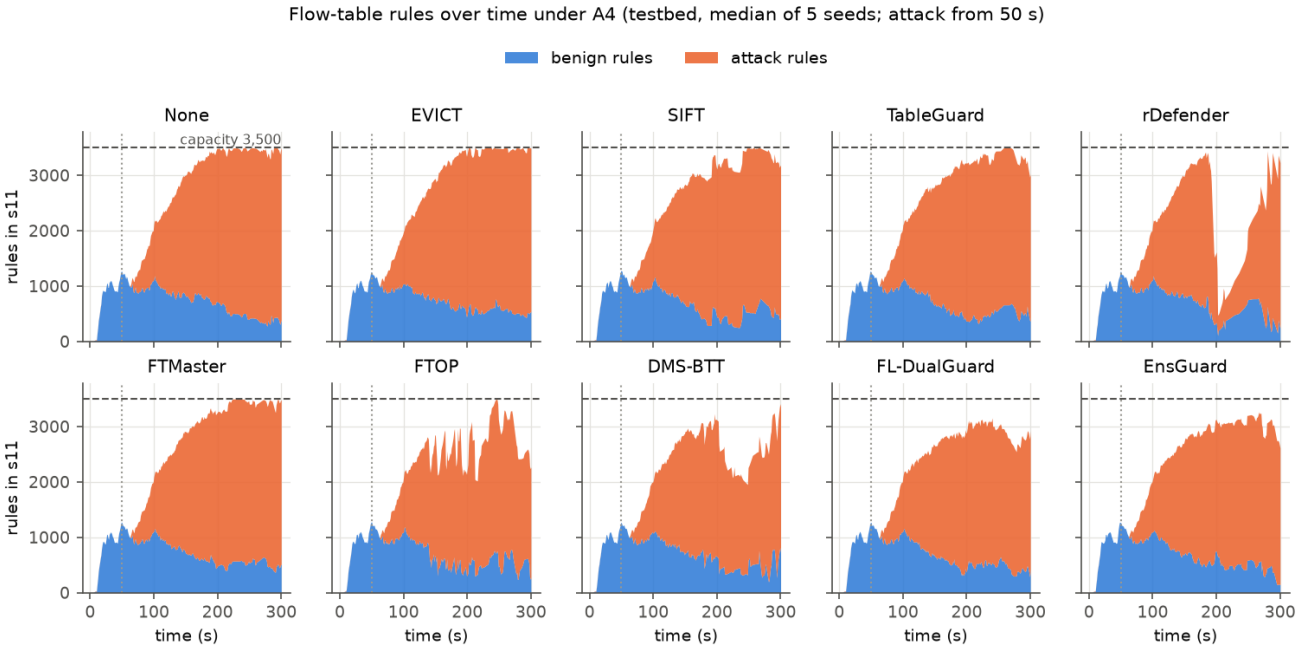


Figure 1 (A4). Normal (blue) and attack (orange) rules in the core switch under the mimicry attack. Every defense slows the attacker a little; rDefender and FTOP clear the table most visibly; EnsGuard keeps the table slightly below full but the attacker holds most of it.

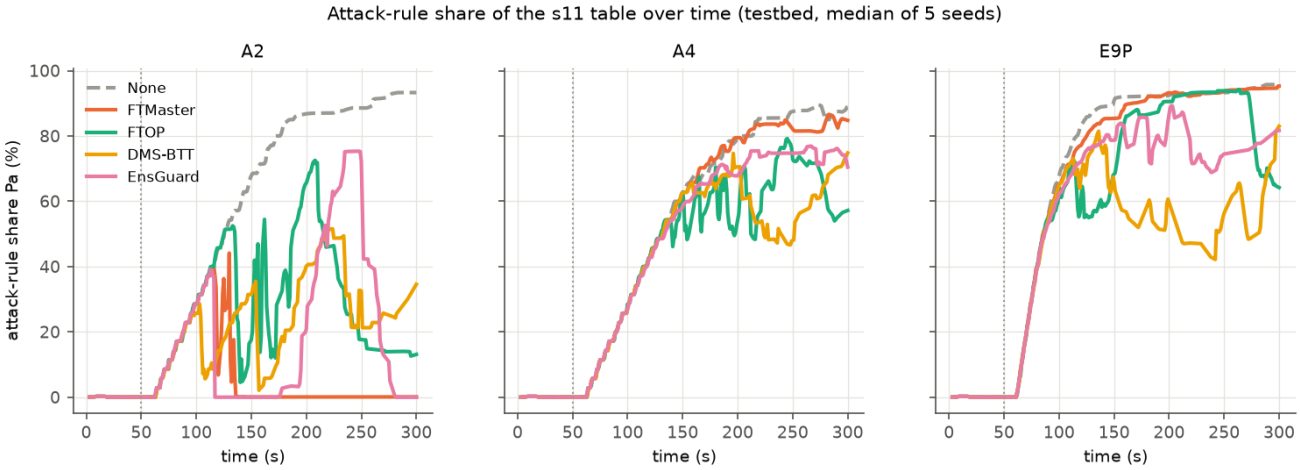


Figure 2. Attacker share over time. Under A2, EnsGuard removes the attacker for long stretches (about 120-190 s and again near 280 s). Under A4 and E9P it stays near 75 %, close to no defense, because the detector does not fire; DMS-BTT does better on E9P (about 50 %).

4.6 What users experience

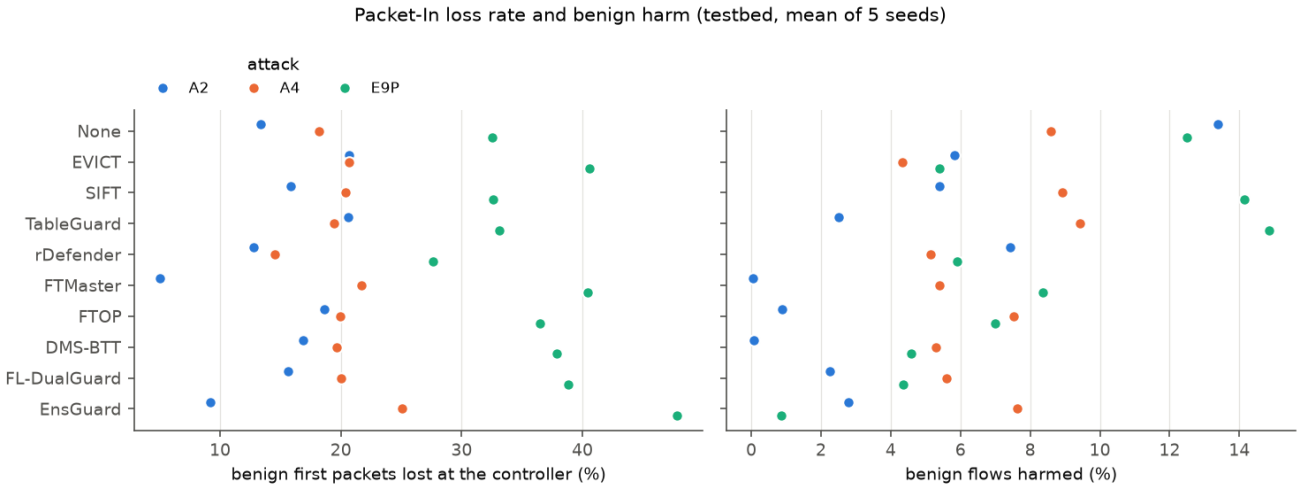


Figure 5. Left: first packets of normal flows lost at the overloaded controller. EnsGuard loses the most under E9P (48 %) because its scoring is slow. Right: normal flows harmed. EnsGuard is the best under E9P (0.9 %) but poor under A4 (7.6 %, against 8.6 % with no defense).

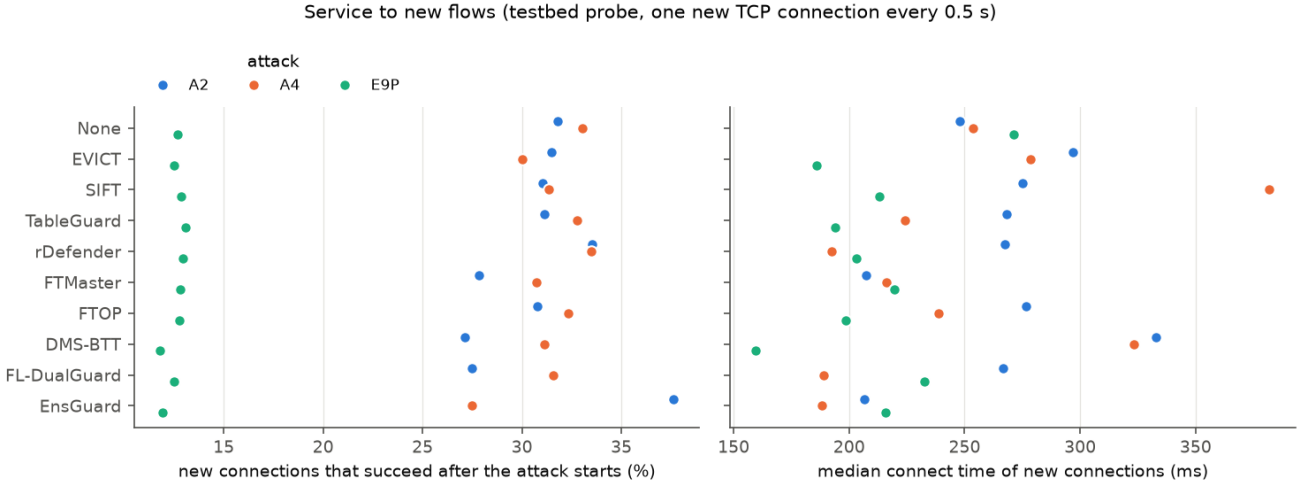


Figure 6. New-connection probe. Under A2, EnsGuard lets the most new connections through (38 %). Under E9P every defense is equally poor (about 12 %): the controller is saturated.

4.7 Cost on the controller

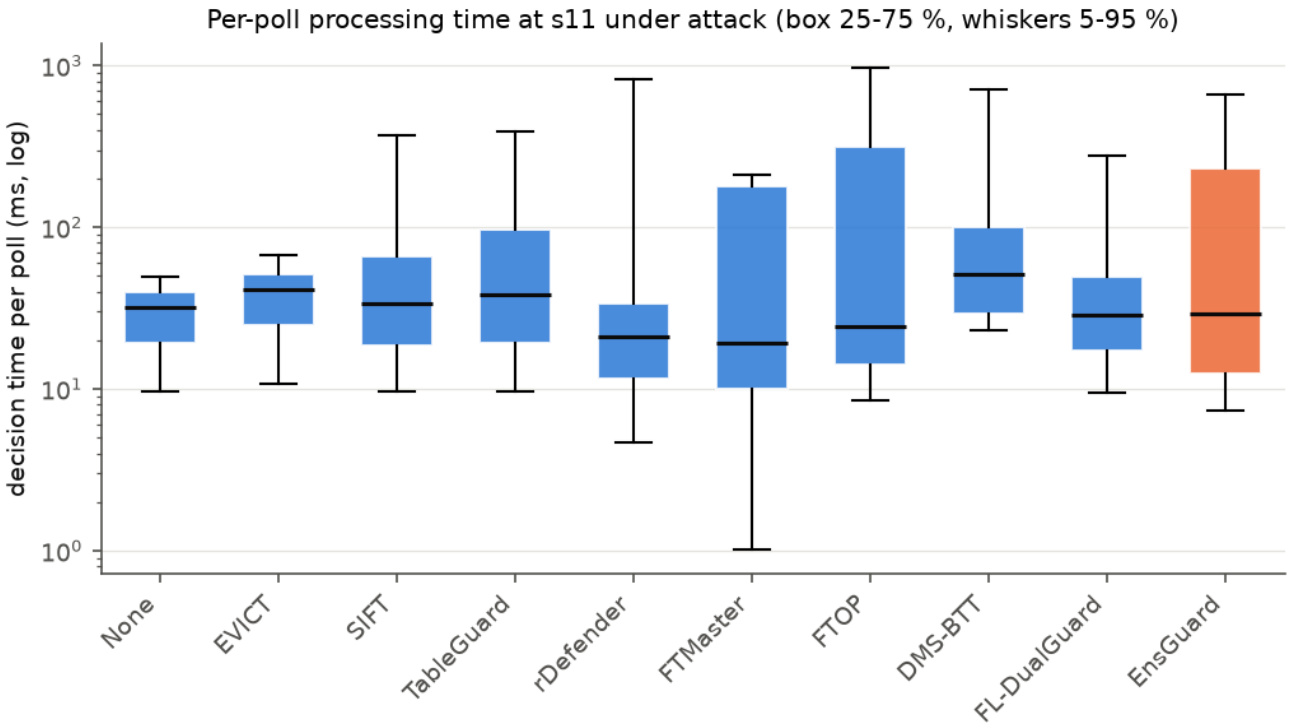


Figure 7. Processing time per poll. EnsGuard is the slowest and most variable (median 135 ms over the three attacks; 160 ms under A4, 232 ms under E9P).

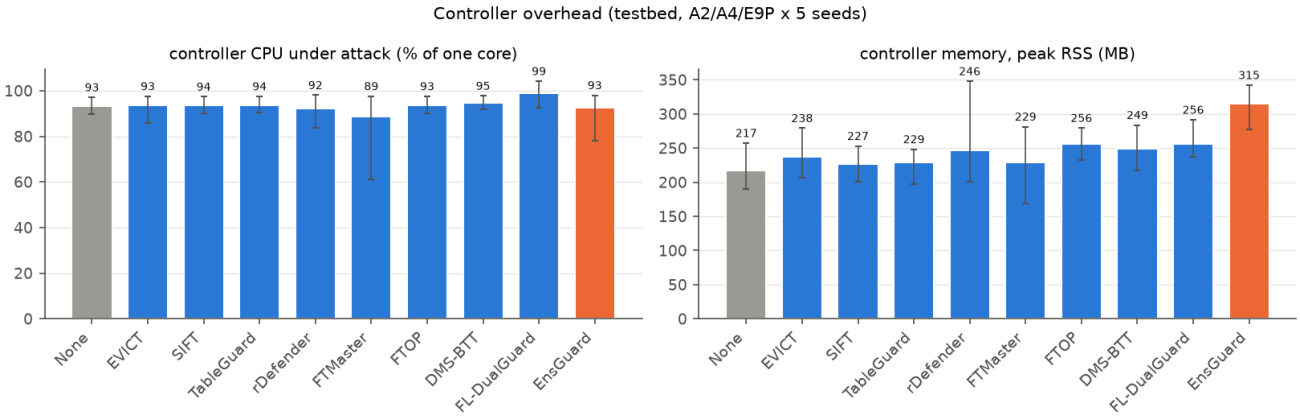


Figure 8. Controller CPU and memory. Every defense runs the controller at about 93 % of one core under attack, so CPU does not separate them; EnsGuard uses the most memory (about 315 MB).

4.8 Why the testbed disagrees with the simulator

We recorded every entry's counters on the testbed in 12 extra runs (logging costs 0.17 ms per poll and did not change the results) and scored every detector on exactly those rows, next to the simulator's version of the same traffic (results/e11_testbed/report.md).

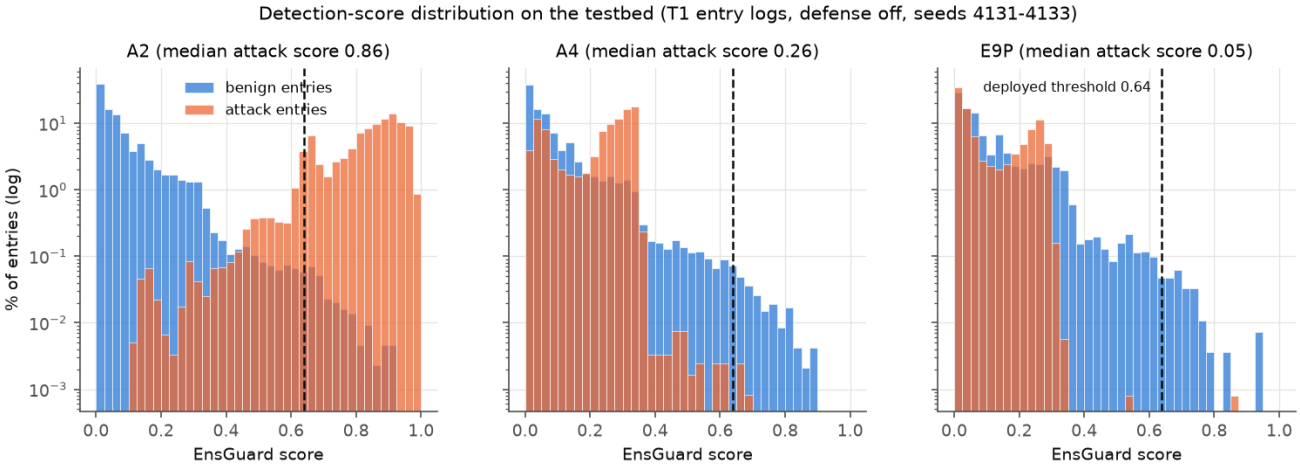


Figure 11. EnsGuard's score for normal (blue) and attack (orange) entries on the testbed. A2's attack entries sit above the 0.64 threshold. A4's and E9P's sit below it, inside the normal traffic's tail, so they are not caught.

- **The testbed reproduces the simulator.** Every detector catches A2 (simulator 81-99 %, testbed 49-95 %) and misses A4 and E9P (at most 1.3 %) in both.
- **The simulator already predicted it.** On the trace's normal traffic, the top 0.1 % of normal scores (0.67) lies above the median A4 score (0.34). On domain D1, which trained on A4, the same detector catches 96.6 % of A4. The cause is a mismatch between the training domains and the real trace (domain shift).
- **Poll jitter is not the cause.** Rebuilding the features on a perfect 1 s clock changes recall very little.
- **Ranking still works, thresholds do not.** On A4 the attack entries still rank high (PR-AUC about 89 for EnsGuard, DMS-BTT and FTOP federated) but below the noisiest normal entries.
- **On the testbed, PortGuard does most of EnsGuard's attacker removal under A4 and E9P.** The block rule's median falls to 0 when few entries are evicted, which is how busy normal hosts got blocked (H4).

4.9 The v4 trial: a preregistered negative result

Hypothesis: if the miss is domain shift, give the trace domain its own controller in the federation, trained on its normal traffic plus the standard attack A2 (never A4 or E9P). Pass/fail rules were written before training (docs/v4_plan.md): A4 recall at least 30 % in the simulator and on the testbed; false alarms at most 0.5 %; A2 recall not lower; unseen-attack recall at least 46.5 %.

Detector	A4 caught (simulator / testbed)	Unseen attacks caught (E11)	Passes
v3 (D1-D3)	0 % / 0 %	48.5 %	no
v4 (trace controller joins, RF + CatBoost)	0 % / 0 %	14.9 %	no
v4, CatBoost only	0 % / 0 %	21.0 %	no

- The new controller's model learned that only A2's tiny fixed packets are an attack, so its vote pushed A4 further toward 'normal' (median A4 score fell from 0.34 to 0.25 in the simulator).
- Because the shared detector is a plain average, a controller that knows only one easy attack dilutes what the others learned: on D1, unseen-attack recall fell from 45.9 % to 0.2 %.
- As fixed in advance we stopped: no retuning and no testbed campaign for this detector. **Lesson:** in federated detection, a client's attack exposure matters as much as its normal data, and plain averaging gives a narrowly trained client a full vote.

5. Where EnsGuard does well, and where it needs work

Does well

What	Evidence
Detecting attacks a controller never trained on (offline)	98.7 % on attacks seen only elsewhere (others 79-82 %); 48.5 % on an attack nobody saw (next best 34.4 %)
The full defense in simulation	harm 0.36-2.0 % per attack; A4 harm 0.73 % against 20.5 % for DMS-BTT with blocklist; no normal host blocked
Beats the classic defenses on the testbed	lower attacker share than EVICT, SIFT, TableGuard in 14 of 15 runs each (H2, p = 0.004)
Quietest control channel	fewest Packet-Ins over the three attacks (61 k) and under A4 and E9P individually
Protects normal users under the adaptive attack E9P	harm 0.9 %, next best 4.4 %
New users under the standard attack A2	38 % of new connections succeed, the most of any defense
Rarely lets the table overflow	second-fewest seconds full and Table-Full messages under A4 and E9P
No side effects without an attack	harm 0.00 %, lost first packets 0.04 % (H3)

Needs work

What	Evidence	Likely cause
No significant win over modern ML defenses on the testbed	H1 supported against none; H2 not against DMS-BTT, FTMaster, FTOP, rDefender, FL-DualGuard	detector misses A4 and E9P on the trace domain
Misses the disguised attacks on the trace domain	A4 and E9P recall 0 % (as for every detector)	domain shift between training domains and real trace
Harm to normal users under A2 and A4	A2 2.8 % (FTMaster 0.04 %); A4 7.6 % (no defense 8.6 %)	evictions of normal entries; blocks of busy hosts
Blocks innocent hosts	4 of 20 runs (H4 fails)	block rule's median falls to 0
Slowest and heaviest	median 160-232 ms per poll under A4/E9P; most memory (315 MB)	six-model ensemble; the RF is about 80 % of the cost
Loses the most first packets under A4 and E9P	25 % and 48 %	slow scoring on a saturated controller

6. Tests we can run next

Ordered by expected value. Every new design choice is made on development seeds and then tested once on fresh seeds under a new preregistration, as for v3 and v4.

#	Test	What it would show	Effort
1	**v4-lite:** keep the D1-D3 detector, switch to CatBoost only, add a floor to the block rule (median at least m0, and a minimum flagged fraction of the host's own entries), score only entries whose counters changed	Fixes H4 (innocent hosts blocked), cuts scoring time to about a fifth and should reduce lost first packets	about 1.5 days incl. a 100-run overnight campaign
2	**Exposure-aware federation:** weight each controller's vote by the attacks it has seen, or let each controller learn a small local combiner over the shared models' scores	Whether federation can help the trace domain without diluting generalisation (the v4 lesson)	offline first, about 0.5 day
3	**Broader attack library for the trace controller:** train it on the public attacks A1-A3, A5, A7 (A4, E9P, A6 still held out)	Whether v4 failed because the controller saw too few attacks	offline, a few hours
4	**Relieve the controller:** an OpenFlow meter limiting Packet-Ins per access port, or the controller on a separate machine	How much of the testbed harm is controller saturation rather than defense quality (affects every defense)	about 1 day
5	**More testbed domains:** the univ2 trace at FTOP's rate (T2) and native rate (T3)	Whether the A4/E9P miss is specific to bigFlows	one overnight run each
6	**Flash crowds** (FTOP's benign-overflow test)	Whether EnsGuard wrongly treats a sudden legitimate surge as an attack	a few hours
7	**Eviction-policy attack (ALFO-style)**	Robustness to an attack that adapts to FIFO / LRU / random eviction	about 1 day (new attack generator)
8	**Three controllers live at once** on two machines	The full federated deployment on real switches, not one domain per run	needs a second machine
9	**More seeds** for the key cells	More statistical power: with 5 seeds a win needs 12 of 15 paired runs	one night per 50 runs

My recommendation. Tests 1 and 4 together give the fairest picture of EnsGuard on the real testbed within about two days. Test 2 is the most interesting research question and can run offline in parallel.

7. How our experiments compare with the related papers

Paper	Attacks tested	Platform	Our equivalent
SFTO-Guard, FTMaster, FTODefender, FTOP, DMS-BTT (Tang et al.)	standard LOFT with different period / step / maximum; FTODefender also spoofed sources	Mininet + Ryu + OVS	A2 (and A5); same platform, same table size and timeout
FloRa	standard LOFT at 4 rates	Mininet + Raspberry Pi	A2
FTShield	standard LOFT plus floods	Tofino P4 hardware	A2
rDefender	advanced low-rate attack replaying real traffic	Dell hardware switch + Mininet	A4
ALFO-Guard	attack adapting to the eviction policy	Mininet + Ryu	E9P (adapts to our defenses' parameters)
LOFT (Cao et al.)	the original attack; no defense	EdgeCore hardware	A2

- On **A2**, the attack the other papers test, every ML defense works on our testbed; FTMaster is best on attacker share and harm, EnsGuard is second on attacker share and best for new connections.
- On **A4 and E9P**, which most papers do not test, every detector fails on the trace domain. This agrees with rDefender and ALFO, which also found that feature-based detectors degrade under disguised and adaptive attacks.
- None of the related papers tests attacks held out from training or seen by a different controller. Our E11 benchmark fills that gap, and it is where EnsGuard clearly leads.
- Our numbers come from our reimplementations on one setup, so they are comparable with each other but not with the numbers printed in the papers (each paper re-runs its baselines under its own setup).

Limitations to mention

- One laptop and one controller: the controller saturates under attack, which affects every defense.
- One trace domain on the testbed; the synthetic domains are simulator-only.
- Reimplemented baselines; attacks from our own generator; 5 test seeds per cell.
- Sections 4.8 and the per-attack breakdowns are exploratory; only 4.3 and 4.9 are preregistered.

Appendix: remaining figures

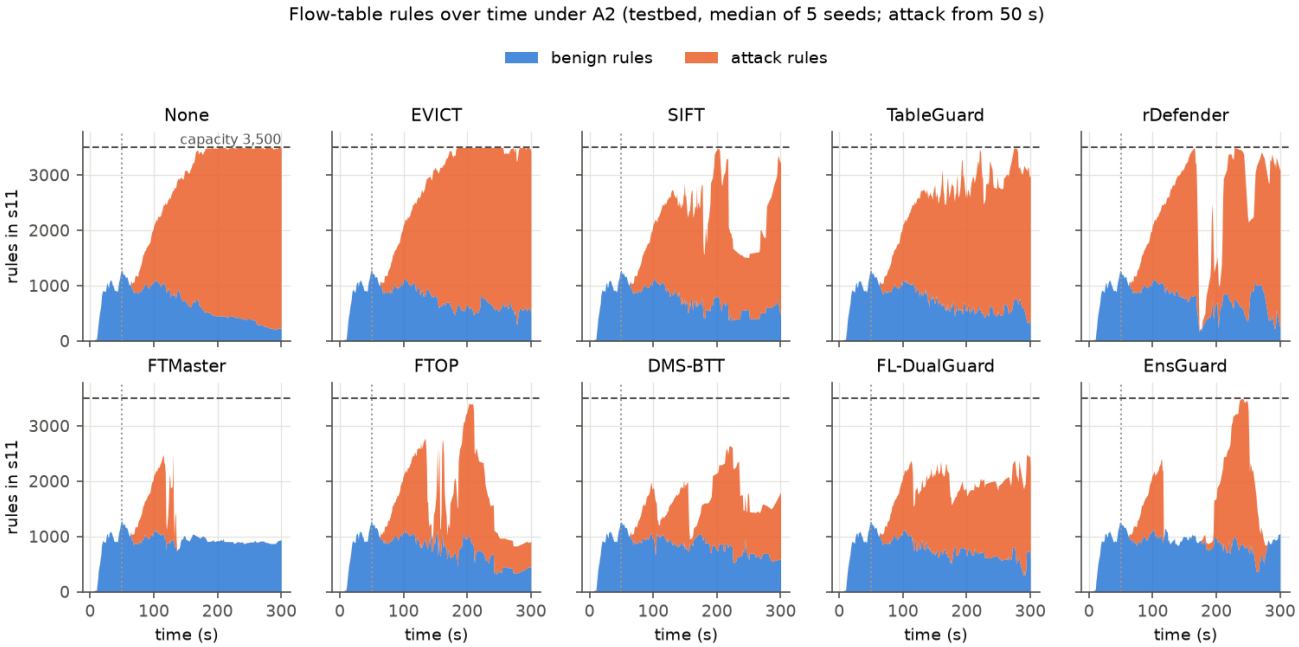


Figure 1 (A2). Rules over time under the standard attack.

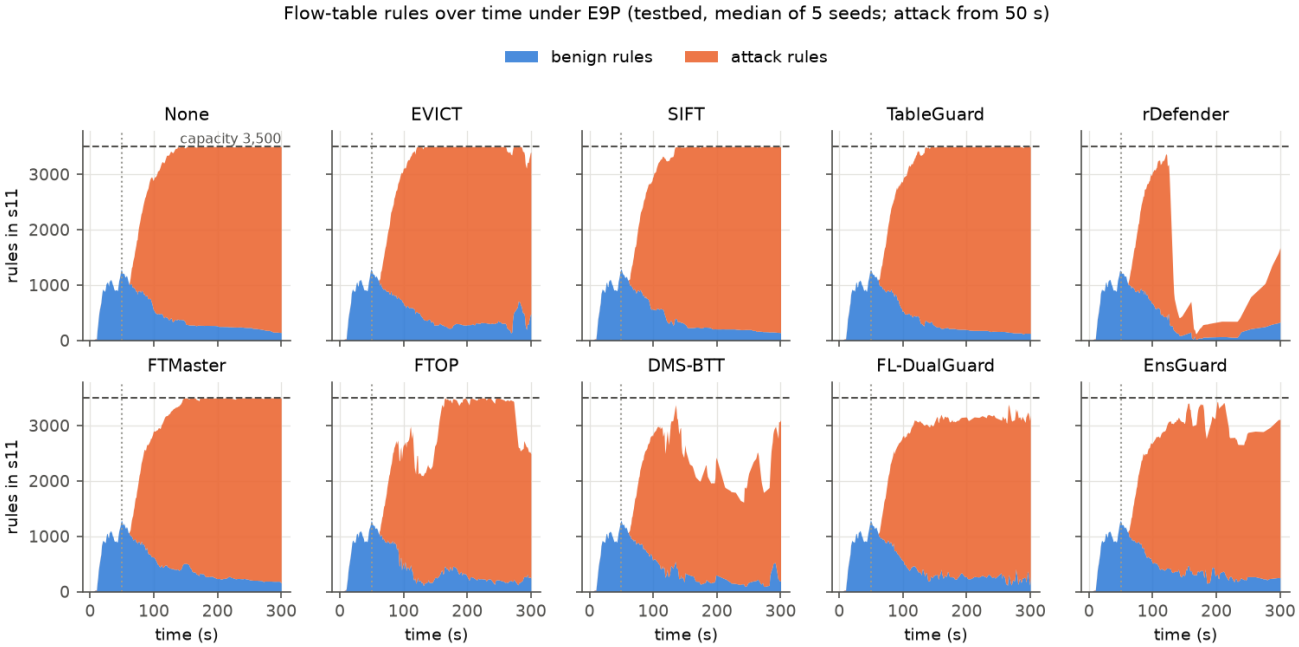


Figure 1 (E9P). Rules over time under the adaptive attack.

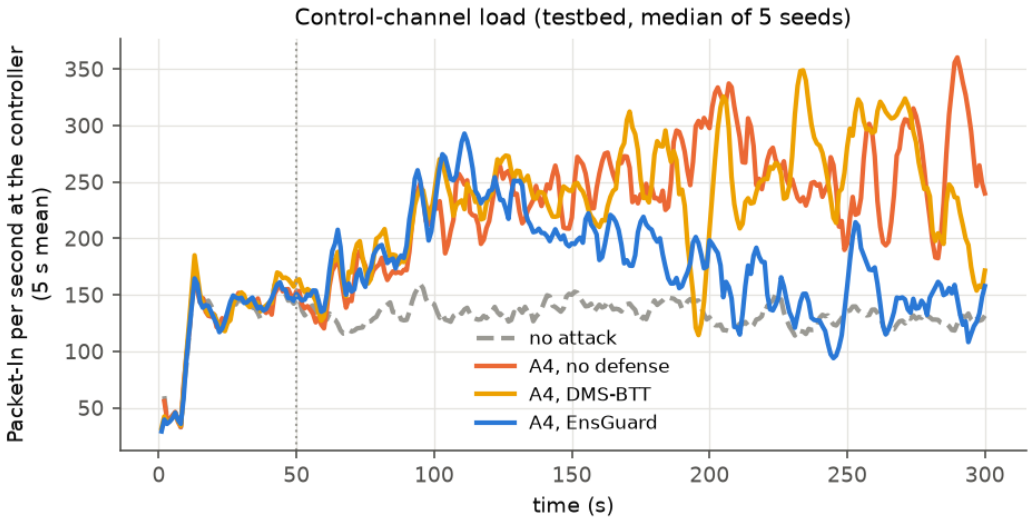


Figure 4. Packet-Ins per second at the controller under A4. With EnsGuard the load returns to near the no-attack level after about 200 s.

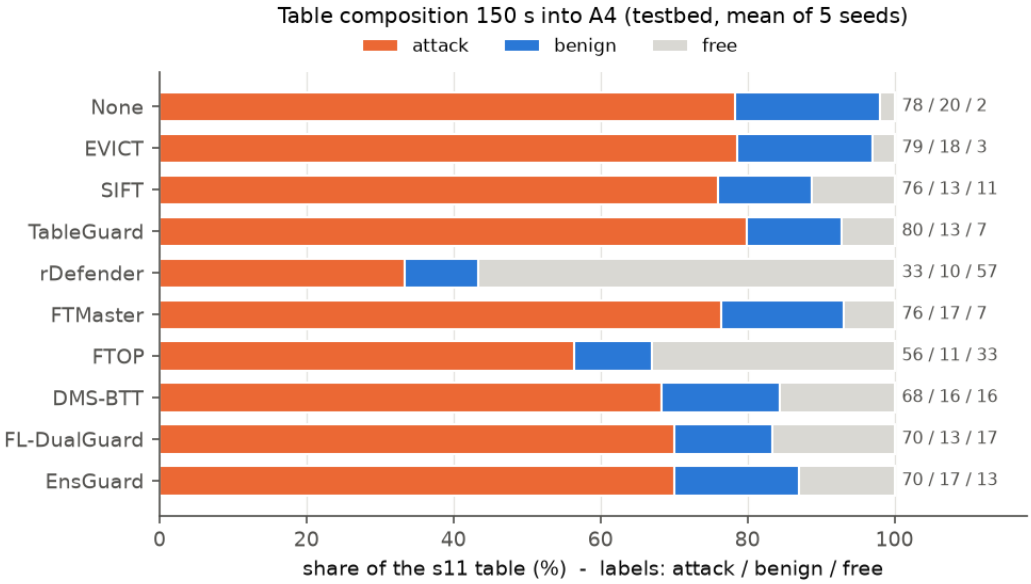


Figure 10. Table composition 150 s into A4 (attack / normal / free). rDefender keeps the most free space.

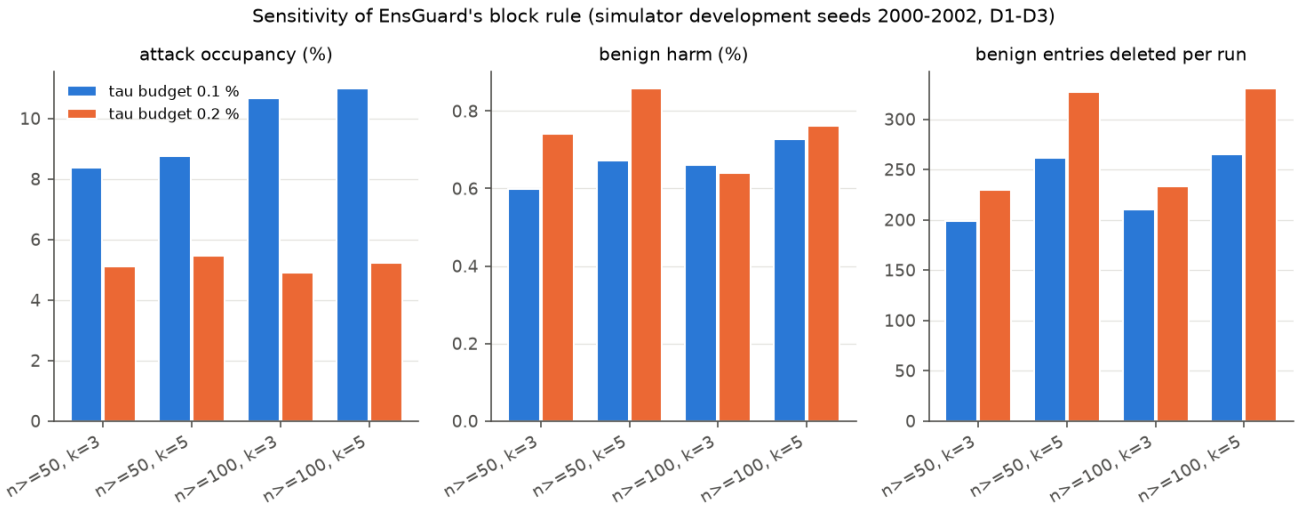


Figure 12. Block-rule settings in the simulator (development seeds). A looser threshold halves the attacker's share at slightly higher harm. On the simulator no setting blocked a normal host; on the testbed the chosen one did.

Glossary

Term	Meaning
Flow table	the switch's list of forwarding rules; ours hold 3,500
Idle timeout	a rule is removed after 10 s without packets
Packet-In	a switch's question to the controller about a packet with no rule
Table-Full	the switch's error message when it cannot add a rule
Attacker share (Pa)	the share of the table held by attack rules
Benign harm	the share of normal flows refused, dropped by a block, or whose first packet was lost
Recall	the share of attack entries the detector flags
False-positive rate	the share of normal entries the detector flags
PR-AUC	how well the detector ranks attack entries above normal ones, independent of the threshold
τ (threshold)	the score above which an entry is evicted; set per controller so 0.1 % of its normal entries would be flagged
Federated / model sharing	controllers exchange trained models, never their traffic
Preregistration	hypotheses, seeds and tests written down before the test runs, so results cannot be tuned afterwards
Holm correction	adjusts p-values when one hypothesis is tested against several comparators
Domain shift	the deployment traffic differs from the training traffic, so learned thresholds no longer fit

Sources: docs/paper_optionA.md (full draft), results/e11_detect, results/evictblock, results/testbed_prereg_v3, results/e11_testbed, results/v4, results/paper_figs. Code: loftbench/ (simulator and detectors), testbed/ (Ryu application and scripts).